

LAB REPORT

Multi Node Clock Synchronization

By: Jakob-Elias Frenzel, Bernhard Bauer

Student Number: es24m016, es24m013

Course: Distributed Embedded Systems

Wien, 2026-05-31

Table of Contents

1	Introduction	4
1.1	Project Context and Objectives	5
1.2	Core Principles	5
1.3	Authorship and Report Structure	5
2	Project Definition	6
2.1	Mission Statement	7
2.2	Core Design Philosophy	7
2.3	Revised Core Constraints	8
2.4	Synchronization Paradigm	8
2.5	Virtual Clock Model	9
2.5.1	Rate Representation	9
2.6	Time Base Definition	9
3	Hardware & Operating Environment	10
3.1	Supported Hardware	11
3.2	GPIO Hardware Interface	11
3.2.1	GPIO 18 — Synchronization Pulse	11
3.2.2	Timing Constraints	11
3.2.3	GPIO 23 — Synchronization Health Indicator	11
3.3	External Verification	12
3.4	Kernel & OS Hardening	12
3.4.1	Required Kernel Parameters	12
3.4.2	Scheduling Requirements	12
3.4.3	CPU Frequency Requirements	12
3.4.4	Memory Requirements	13
3.4.5	Time Service Isolation	13
4	Network Architecture	13
4.1	Network Topology	14
4.2	IP Addressing	14
4.3	Discovery Mechanism	14
4.4	Security Assumptions	14
5	DRS Wire Protocol	15
5.1	Packet Format	16
5.2	Packet Size	16
5.3	Packet Layout	16
5.4	Message Types	16

5.5	Flags Field	17
5.6	Timestamp Semantics	17
5.7	Timestamping Mechanism	17
5.8	CRC32	17
6	Synchronization Mathematics & Control	17
6.1	Offset Estimation	18
6.2	Round Trip Delay	18
6.3	Min-Delay Filter	18
6.4	Automated Calibration	18
6.5	Dual-Loop PI Clock Discipline	19
6.5.1	Step Mode	19
6.5.2	Slew Mode	19
6.6	PI Controller	19
6.7	Slew Rate Limit	20
7	Leader Election & State Machine	20
7.1	Election Model	21
7.2	Node States	21
7.3	GROUND State	21
7.4	Heartbeats	21
7.5	Immediate Demotion	21
7.6	HOLDOVER Mode	22
8	Real-Time Scheduling	22
9	Failure Recovery	23
9.1	Retry Storm Protection	24
9.2	Filter Reset Conditions	24
9.3	Sequence Wraparound	24
10	Lock-Free Clock Access	24
11	Functional Requirements	25
12	Non-Functional Requirements	26
13	systemd Deployment	27
14	Performance Targets	28
15	Architectural Constraints	29
16	Final System Goal	30
17	DRS Telemetry Packet Format	31
17.1	Transport	32
17.2	Packet Layout	32
17.3	State Values	32
17.4	Rate Field (Q32.32 Fixed-Point)	32
17.5	Python Unpack Snippet	33

18 DRS Development Workflow	33
18.1 Two-Loop Architecture	34
18.1.1 Inner Loop — WSL2, x86_64 (seconds per cycle)	34
18.1.2 Outer Loop — RPi via SSH (minutes per cycle)	34
18.2 Project Structure	34
18.3 HAL Stubs (Enable Host Testing)	35
18.4 What Unit Tests Cover (WSL2, no hardware needed)	35
18.5 What RPi Tests Cover	35
18.6 Telemetry Design	35
18.7 Build Commands (run from Windows via <code>wsl</code> ...)	36
18.7.1 Inner loop	36
18.7.2 Outer loop	36
18.8 Implementation Order & Commit Strategy	36
18.9 Verification Checklist	37

1 Introduction

This project was developed as part of the **Distributed Embedded Systems (DES)** class.

1.1 Project Context and Objectives

The primary goal of this project is to build a Distributed Real-Time System (DRS) application capable of establishing a highly precise global time base across a dynamic cluster of hardware nodes (such as Raspberry Pis). The nodes communicate over mixed networks, including both Ethernet and Wi-Fi, utilizing UDP sockets in C/C++.

The overarching objective is to achieve a synchronization precision of **< 100 μ s** between the nodes. Crucially, the synchronization protocol is designed to run entirely in the user-space of a standard Linux OS without the need for custom kernel drivers.

1.2 Core Principles

The system architecture is guided by several core engineering requirements:

- **KISS Principle (Keep It Simple, Stupid):** The system requires zero manual configuration regarding network addresses or node roles. When a new node is plugged in, it integrates seamlessly.
- **Autonomous Discovery:** Nodes discover each other dynamically without a centralized registry.
- **Resilience:** The protocol dynamically filters latency jitter (especially asymmetrical jitter introduced by Wi-Fi) and is resilient to crash failures, ensuring there is no single point of failure if a leader node crashes.
- **Verifiability:** The claimed synchronization precision must be objectively measurable and falsifiable using external verification tools to prove physical simultaneity, rather than relying solely on internal software timestamps.

1.3 Authorship and Report Structure

It is important to note that both the codebase for this project and the generation of this report were heavily assisted by Artificial Intelligence (AI). The underlying system architecture and engineering requirements, however, were carefully thought out and designed by the architecture team.

The remainder of this report is structured to walk through these architectural decisions in detail. The subsequent chapters outline the complete system architecture—ranging from the hardware environment and network topology to the specific wire protocol and synchronization

mathematics. Finally, the report concludes with an overview of the system's telemetry data formatting and the development workflow used to build and verify the cluster.

2 Project Definition

2.1 Mission Statement

The DRS project establishes a deterministic, high-precision distributed global time base across a dynamic cluster of Raspberry Pi nodes operating exclusively in Linux user space.

The system shall maintain a physical synchronization delta of:

$\Delta t_{\text{physical}} < 100 \mu\text{s}$

as measured externally using GPIO-generated synchronization pulses observed on a logic analyzer or oscilloscope.

The architecture prioritizes:

- deterministic behavior
- operational simplicity
- fail-safe recovery
- real-time scheduling integrity
- minimal protocol complexity
- bounded timing behavior

2.2 Core Design Philosophy

The synchronization subsystem SHALL follow the KISS principle:

Keep It Simple, Stupid (KISS)

The protocol intentionally avoids:

- distributed consensus frameworks
- dynamic spanning trees
- kernel modifications
- external time authorities
- heavyweight serialization
- multi-hop routing
- dynamic routing protocols
- cryptographic processing in the synchronization hot path

The synchronization cluster is designed exclusively for:

- single-hop Layer-2 Gigabit Ethernet
- trusted laboratory environments
- fail-stop node behavior
- deterministic LAN conditions

2.3 Revised Core Constraints

- Wired Gigabit Ethernet SHALL be the exclusive synchronization transport.
- WLAN MAY be used exclusively for out-of-band management and provisioning.
- CPU Core 3 SHALL be reserved exclusively for synchronization operations.
- WLAN interrupts SHALL NEVER execute on Core 3.
- Ethernet IRQs SHALL NEVER execute on Core 3.
- The Linux host system clock SHALL NEVER be modified.
- All synchronization adjustments SHALL occur exclusively inside the virtual clock layer.

Explicitly prohibited:

- `clock_settime()`
- `adjtimex()`
- `CLOCK_REALTIME` stepping
- kernel PLL interaction

2.4 Synchronization Paradigm

The system implements:

Internal Monotonic Synchronization

A single elected leader defines the cluster reference timeline.

Follower nodes estimate:

- phase offset
- frequency drift
- path delay

relative to the elected leader.

The synchronization domain is entirely internal.

The cluster SHALL NOT synchronize to:

- UTC
- NTP

- GPS
- PTP grandmasters
- wall-clock time

2.5 Virtual Clock Model

The virtual clock mapping SHALL be defined as:

$$T_{\text{global}} = (T_{\text{local_raw}} \times \text{Rate}) + \text{Offset} - \text{LatencyCorrection_ns}$$

Where:

Parameter	Description
$T_{\text{local_raw}}$	CLOCK_MONOTONIC_RAW timestamp
Rate	Fixed-point frequency scaling multiplier
Offset	Phase correction in nanoseconds
LatencyCorrection_ns	Static calibration compensation

2.5.1 Rate Representation

Rate SHALL be represented as signed 64-bit Q32.32 fixed-point.

Definitions:

Value	Meaning
0x00000001_00000000	1.0x nominal clock rate
+1000 ppm	Maximum positive slew
-1000 ppm	Maximum negative slew

Floating-point arithmetic SHALL NOT be used inside the synchronization hot path.

2.6 Time Base Definition

The synchronization engine SHALL operate exclusively on:

CLOCK_MONOTONIC_RAW

All protocol timestamps SHALL represent:

Absolute nanoseconds since local kernel boot.

The cluster timeline SHALL be defined relative to the elected leader's monotonic boot-time origin.

The system SHALL NOT use:

- Unix Epoch
- wall-clock time
- CLOCK_REALTIME

Optional wall-clock export layers MAY exist externally but remain outside synchronization scope.

3 Hardware & Operating Environment

3.1 Supported Hardware

Target platform:

- Raspberry Pi 4B
- Quad-core Cortex-A72
- Gigabit Ethernet
- PREEMPT_RT Linux kernel

Recommended:

- passive cooling
- fixed thermal conditions
- wired Ethernet only

3.2 GPIO Hardware Interface

3.2.1 GPIO 18 — Synchronization Pulse

GPIO 18 SHALL generate:

- a 10 ms HIGH pulse
- once per global second

The rising edge SHALL define the global second boundary.

The pulse SHALL remain active in all operational states except:

GROUND

3.2.2 Timing Constraints

| Property | Requirement | |—|—| | Max pulse edge jitter | <20 μ s | | Pulse source | Virtual global clock only | | Pulse generation thread | RT synchronization thread |

3.2.3 GPIO 23 — Synchronization Health Indicator

GPIO 23 SHALL indicate synchronization health.

| State | Meaning | |—|—| | LOW | Stable synchronized operation | | HIGH | Startup, degraded sync, recovery, calibration, or fault |

GPIO 23 SHALL transition LOW only after:

`|Offset| < 100 µs`

for a continuous:

`10-second interval`

3.3 External Verification

Recommended instrumentation:

- multi-channel digital oscilloscope
- logic analyzer

Minimum sample rate:

`≥ 1 MHz`

Measurement criteria:

- physical delta between GPIO 18 rising edges

3.4 Kernel & OS Hardening

3.4.1 Required Kernel Parameters

```
isolcpus=3
nohz_full=3
rcu_nocbs=3
```

3.4.2 Scheduling Requirements

Synchronization thread SHALL:

- execute exclusively on Core 3
- use `SCHED_FIFO`
- use priority 85

3.4.3 CPU Frequency Requirements

CPU governor SHALL be:

`performance`

Turbo and dynamic voltage scaling SHOULD be disabled.

3.4.4 Memory Requirements

All synchronization process memory SHALL be locked using:

```
mlockall(MCL_CURRENT | MCL_FUTURE)
```

No page faults SHALL occur in the synchronization hot path.

3.4.5 Time Service Isolation

The following services SHALL be disabled:

- systemd-timesyncd
- chronyd
- ntpd
- ptp4l
- phc2sys

4 Network Architecture

4.1 Network Topology

Topology assumptions:

- single-hop Ethernet
- switched Gigabit LAN
- no routing
- no NAT
- no Wi-Fi bridges
- standard MTU 1500
- no jumbo frames

4.2 IP Addressing

Static schema:

10.0.0.XY

Field	Meaning	— —	X	Team ID (1–8)	Y	Node ID (1–3)
-------	---------	-------	---	---------------	---	---------------

Example:

Team 4 Node 2 → 10.0.0.42

4.3 Discovery Mechanism

Discovery SHALL use UDP multicast.

Parameter	Value	— —	Group	239.192.88.100	Port	47200	IP_MULTICAST_LOOP
							0

IP_MULTICAST_LOOP = 0 SHALL disable sender-local multicast loopback only.

4.4 Security Assumptions

The synchronization protocol assumes:

- trusted LAN operation
- fail-stop node behavior

- bounded congestion
- non-malicious traffic

The protocol provides NO:

- authentication
- encryption
- replay protection
- Byzantine fault tolerance
- confidentiality guarantees

Deployment outside trusted laboratory environments is NOT supported.

5 DRS Wire Protocol

5.1 Packet Format

All packets SHALL:

- use fixed-size binary encoding
- use network byte order (Big Endian)
- avoid dynamic allocation in the hot path
- avoid text serialization

Raw struct casting is prohibited.

All fields SHALL be serialized explicitly using:

- `htons()`
- `htonl()`
- explicit 64-bit endian conversion

5.2 Packet Size

All packets SHALL be exactly:

64 bytes

5.3 Packet Layout

Field	Type	Size	Description
Magic	uint32	4	Constant: 0x44525354 ("DRST")
Version	uint8	1	Protocol version
MsgType	uint8	1	Message type
Flags	uint8	1	Synchronization state flags
Reserved	uint8	1	Reserved
Seq	uint16	2	Rolling sequence number
NodeID	uint32	4	Unique node ID
ElectionTerm	uint32	4	Leader election epoch
T1	uint64	8	Follower transmit timestamp
T2	uint64	8	Leader receive timestamp
T3	uint64	8	Leader transmit timestamp
T4	uint64	8	Follower receive timestamp
CRC32	uint32	4	Packet integrity
Padding	uint8[12]	12	Reserved
Total	—	64	Total packet size

5.4 Message Types

Value	Meaning
0x01	ANNOUNCE
0x02	SYNC_REQ
0x03	SYNC_RESP

5.5 Flags Field

Bit	Meaning
0	LEADER
1	HOLDOVER
2	CALIBRATED
3	FAULT
4-7	Reserved

Reserved bits SHALL be zero.

5.6 Timestamp Semantics

Timestamp	Meaning
T1	Sync request send time
T2	Kernel-captured receive timestamp
T3	Kernel-captured transmit timestamp
T4	Sync response receive timestamp

All timestamps SHALL use:

CLOCK_MONOTONIC_RAW nanoseconds

5.7 Timestamping Mechanism

Mandatory Linux API:

SO_TIMESTAMPING

Accepted modes:

1. hardware NIC timestamping
2. kernel software timestamping

The highest precision mode supported by the platform SHALL be selected automatically.

5.8 CRC32

CRC32 SHALL use:

- IEEE 802.3 polynomial 0x04C11DB7
- initial value 0xFFFFFFFF
- reflected input and output

Reserved padding bytes SHALL:

- be zero-filled
- be included in CRC validation

6 Synchronization Mathematics & Control

6.1 Offset Estimation

$$\theta = ((T2 - T1) + (T3 - T4)) / 2$$

6.2 Round Trip Delay

$$\delta = (T4 - T1) - (T3 - T2)$$

6.3 Min-Delay Filter

The synchronization engine SHALL maintain:

- rolling RTT window size = 10

Only samples within:

$$\text{minimum RTT} + 10 \mu\text{s}$$

SHALL be accepted.

Rejected samples include:

- scheduler spikes
- interrupt storms
- retransmission artifacts
- congestion bursts

6.4 Automated Calibration

Each node SHALL perform self-latency calibration before entering synchronization states.

Calibration SHALL:

- use SO_TIMESTAMPING loopback
- use minimum 50 samples
- use minimum-delay selection

- reject outliers >20 μ s from minimum

Calibration output SHALL define:

LatencyCorrection_ns

Calibration SHALL re-run:

- during startup
- after leader election
- after HOLDOVER expiration
- after synchronization fault recovery

6.5 Dual-Loop PI Clock Discipline

6.5.1 Step Mode

A hard phase step SHALL require:

3 consecutive samples confirming $|\theta| > 1$ ms

Only the virtual clock MAY step.

The host OS clock SHALL NEVER be modified.

6.5.2 Slew Mode

If:

$|\theta| \leq 1$ ms

then gradual frequency correction SHALL occur through PI control.

6.6 PI Controller

$\text{Rate_new} = \text{Rate_old} + K_p \cdot \theta + K_i \cdot \int \theta dt$

Recommended defaults:

| Parameter | Value | |—|—| | Kp | 0.05 | | Ki | 0.005 | | Update Period | 50 ms |

Integrator SHALL be clamped to:

± 1000 ppm equivalent

6.7 Slew Rate Limit

Maximum correction rate:

1000 ppm

Equivalent to:

50 μ s per 50 ms cycle

7 Leader Election & State Machine

7.1 Election Model

The system implements a modified Bully algorithm.

Lowest NodeID SHALL win leadership.

Election timeout SHALL be randomized within:

250–500 ms

to reduce election collisions.

7.2 Node States

| State | Description | |—|—| | GROUND | Startup stabilization | | CALIBRATION | Self-latency measurement | | LISTEN | Passive discovery | | CANDIDATE | Election in progress | | FOLLOWER | Synchronized to leader | | LEADER | Authoritative clock source | | HOLDOVER | Temporary freerun mode | |

7.3 GROUND State

After boot:

- filters SHALL be zeroed
- transmissions SHALL be suppressed
- multicast listening SHALL remain active

Duration:

2 seconds

7.4 Heartbeats

| Property | Value | |—|—| | Leader announce interval | 100 ms | | Follower timeout | 3 missed heartbeats | | Sync exchange period | 50 ms |

7.5 Immediate Demotion

If a lower-ID node appears:

- current leader SHALL demote immediately
- election SHALL restart deterministically

The lower-ID node SHALL assume leadership only after:

- successful calibration
- valid heartbeat transmission

7.6 HOLDOVER Mode

If leader communication is lost:

- node SHALL enter HOLDOVER
- last stable Rate SHALL be maintained
- Offset SHALL be frozen
- GPIO 23 SHALL transition HIGH
- drift adaptation SHALL stop

Maximum HOLDOVER duration:

10 seconds

After expiration:

- node SHALL re-enter election

8 Real-Time Scheduling

Synchronization thread SHALL:

- use SCHED_FIFO
- use priority 85
- remain pinned to Core 3

The synchronization loop MUST periodically yield using:

```
clock_nanosleep()
```

to prevent:

- softirq starvation
- scheduler deadlock
- SSH lockout

9 Failure Recovery

9.1 Retry Storm Protection

Failed nodes SHALL reduce multicast rate to:

5 Hz

after:

5 consecutive synchronization failures

9.2 Filter Reset Conditions

Synchronization filters SHALL reset on:

- reboot
- sequence discontinuity
- leader change
- timestamp overflow
- phase correction >5 ms

9.3 Sequence Wraparound

Sequence numbers SHALL use unsigned 16-bit rollover semantics.

A sequence transition:

65535 → 0

SHALL be treated as valid wraparound.

Backward jumps exceeding:

1024 sequence values

SHALL invalidate synchronization state.

10 Lock-Free Clock Access

The virtual clock SHALL support lock-free concurrent readers.

The synchronization thread SHALL be the exclusive writer.

Reader access SHALL:

- avoid mutex blocking
- avoid dynamic allocation
- guarantee monotonic reads

Offset and Rate updates SHALL use:

- atomic 64-bit operations
- acquire/release memory ordering

11 Functional Requirements

| ID | Requirement | |—|—| | F-1 | Autonomous multicast discovery | | F-2 | Virtual global monotonic clock | | F-3 | Min-delay filtering | | F-4 | Deterministic bully election | | F-5 | Explicit fault recovery mapping | | F-6 | Lock-free telemetry | | F-7 | Stable convergence detection | | F-8 | Immediate leader demotion | | F-9 | Non-blocking writer precedence | | F-10 | Monotonic nanosecond representation | | F-11 | GPIO validation pulse | | F-12 | Fixed performance governor | | F-13 | SO_TIMESTAMPING support | | F-14 | Automatic filter reset | | F-15 | Fixed 64-byte packets | | F-16 | 100 ms synchronization timeout | | F-17 | Step vs slew discipline | | F-18 | Fail-silent GPIO indicator | | F-19 | Seamless late joiners | | F-20 | Deterministic isolated startup | | F-21 | Static latency calibration | | F-22 | Retry-storm suppression | | F-23 | IRQ affinity isolation | | F-24 | Explicit endian-safe serialization | | F-25 | Holdover freerun support | | F-26 | CRC packet validation | | F-27 | Sequence wraparound handling | | F-28 | Integral windup prevention | | F-29 | Step confirmation hysteresis | | F-30 | Convergence signaling | | F-31 | Automated calibration | | F-32 | Calibration-on-election |

12 Non-Functional Requirements

ID	Requirement
NF-1	Physical pulse delta <100 μ s
NF-2	Pure user-space implementation
NF-3	Jitter resilience under mixed traffic
NF-4	Lock convergence within 10 s
NF-5	3-heartbeat hysteresis
NF-6	Writer never blocked by readers
NF-7	SCHED_FIFO priority 85
NF-8	mlockall memory locking
NF-9	No hot-path disk or console I/O
NF-10	Isolated Core 3 execution
NF-11	Max slew = 1000 ppm
NF-12	No dynamic heap allocation in hot path
NF-13	No packet fragmentation
NF-14	Deterministic packet parsing
NF-15	No mutex contention in sync loop
NF-16	Core 3 sanctity

13 systemd Deployment

Example:

```
[Unit]
Description=DRS Synchronization Service
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/local/bin/drs_sync
Restart=on-failure
RestartSec=5s
CPUAffinity=3
CPUSchedulingPolicy=fifo
CPUSchedulingPriority=85
LimitRTPRIO=95
LimitMEMLOCK=infinity
NoNewPrivileges=true

[Install]
WantedBy=multi-user.target
```

14 Performance Targets

Component	Budget		
NIC timestamp jitter	<20 μ s		
Scheduler latency	<30 μ s		
Filter residual	<20 μ s		
GPIO generation	<20 μ s		
Total expected worst-case	<100 μ s		

15 Architectural Constraints

The following are explicitly prohibited:

- modifying kernel clocks
- custom kernel modules
- TCP transport
- Wi-Fi synchronization operation
- distributed consensus protocols
- mutex blocking in hot path
- filesystem I/O in synchronization loop
- floating-point exceptions in RT thread

16 Final System Goal

The DRS cluster SHALL behave as a single deterministic distributed timing domain where all participating nodes produce externally measurable synchronization pulses aligned within:

$<100\ \mu\text{s}$

under:

- PREEMPT_RT Linux
- standard user-space execution
- single-hop Ethernet
- dynamic node membership
- moderate background traffic conditions

17 DRS Telemetry Packet Format

17.1 Transport

- **Protocol:** UDP unicast
- **Port:** 4242
- **Direction:** each Pi → visualizer machine IP (passed as second CLI arg to `drs_sync`)
- **Rate:** one packet per sync exchange (follower: ~10 Hz, leader: ~20 Hz at 50 ms tick)

17.2 Packet Layout

Fixed 40-byte binary, **little-endian** (native RPi byte order, no byte-swap).

Offset	Size	Type	Field	Description
0	8	int64	timestamp_ns	CLOCK_MONOTONIC_RAW on the sending node (nanoseconds)
8	4	int32	state	Node state enum (see below)
12	8	int64	offset_ns	Measured clock offset from leader (nanoseconds). 0 for leader packets.
20	8	int64	rtt_ns	Round-trip time of the sync exchange (nanoseconds). 0 for leader packets.
28	8	int64	rate_q32	Virtual clock rate as Q32.32 fixed-point (see below)
36	4	uint32	node_id	Sender's node ID (matches last octet of its IP: 10.0.0.<node_id>)

17.3 State Values

Value	Name	Meaning
0	GROUND	Startup hold-off (waiting ~1 s before participating)
1	CALIBRATION	Measuring self-latency (one-shot at startup)
2	LISTEN	Waiting to hear a leader before running for election
3	CANDIDATE	Running for election (transitions to LEADER immediately if no lower-ID node seen)
4	FOLLOWER	Synchronized to leader — offset_ns and rtt_ns are valid
5	LEADER	This node is the time reference — offset_ns and rtt_ns are always 0
6	HOLDOVER	Leader lost; free-running on last known rate for up to 10 s

17.4 Rate Field (Q32.32 Fixed-Point)

`rate_q32` is a signed 64-bit integer. The integer value $2^{32} = 4294967296$ represents a rate of exactly 1.0 (nominal, no adjustment).

To convert to a human-readable PPM deviation:

```
RATE_ONE = 1 << 32 # 4294967296
ppm = (rate_q32 - RATE_ONE) / RATE_ONE * 1_000_000
```


Normal range: ± 1000 ppm (clamped by the PI controller).

17.5 Python Unpack Snippet

```
import struct

RECORD_SIZE = 40

STATE_NAMES = {
    0: "GROUND", 1: "CALIBRATION", 2: "LISTEN",
    3: "CANDIDATE", 4: "FOLLOWER", 5: "LEADER", 6: "HOLDOVER"
}

def unpack(data: bytes) -> dict:
    ts, state, offset, rtt, rate, node_id = struct.unpack_from("<qiqqqI", data)
    return {
        "node_id": node_id,
        "timestamp_ns": ts,
        "state": STATE_NAMES.get(state, state),
        "offset_ns": offset,
        "offset_us": offset / 1000,
        "rtt_ns": rtt,
        "rtt_us": rtt / 1000,
        "rate_ppm": (rate - (1 << 32)) / (1 << 32) * 1e6,
    }
```

Note: `timestamp_ns` is `CLOCK_MONOTONIC_RAW` from each node's local clock — it is **not** comparable between nodes. Use `offset_ns` to understand inter-node alignment.

18 DRS Development Workflow

18.1 Two-Loop Architecture

Development runs on Windows with WSL2 as the build environment and one Raspberry Pi 4B available over SSH.

18.1.1 Inner Loop — WSL2, x86_64 (seconds per cycle)

Edit → compile → unit test → iterate

All platform-independent logic is unit-tested here without hardware. This is where the agentic loop spends most of its time.

18.1.2 Outer Loop — RPi via SSH (minutes per cycle)

Cross-compile → deploy → run → read telemetry → iterate

Hardware-specific behavior (SO_TIMESTAMPING, SCHED_FIFO, GPIO) is validated on the actual target.

18.2 Project Structure

```
DES_ClockSync/
├── src/
│   ├── main.c
│   ├── clock.h / clock.c      # Virtual clock: Q32.32 rate/offset, atomics
│   ├── protocol.h / protocol.c # 64-byte packet: serialize, deserialize, CRC32
│   ├── sync.h / sync.c        # PI controller, min-delay filter, T1-T4 math
│   ├── election.h / election.c # State machine, Bully algorithm
│   ├── gpio.h / gpio.c        # GPIO 18/23 (HAL – stubbed on host build)
│   ├── net.h / net.c          # UDP socket, SO_TIMESTAMPING, multicast
│   └── telemetry.h / telemetry.c # Lock-free ring buffer + UDP drain thread
├── tests/
│   ├── main_test.c            # Test runner (no external deps)
│   ├── test_protocol.c        # Serialize/deserialize, CRC32, endian
│   ├── test_clock.c           # Q32.32 math, virtual clock formula
│   ├── test_sync.c            # PI controller, min-delay filter, T1-T4 calc
│   └── test_election.c         # State machine transitions
├── scripts/
│   ├── deploy.sh              # scp binary + systemd restart on RPi
│   └── telemetry_listen.py     # Windows-side UDP listener (prints CSV to stdout)
├── CMakeLists.txt
└── toolchain-aarch64.cmake    # Cross-compile: aarch64-linux-gnu-gcc
```

18.3 HAL Stubs (Enable Host Testing)

Three functions are conditionally compiled via `-DPLATFORM=host`:

Function	Host (WSL2)	RPi
<code>gpio_set(pin, val)</code>	no-op	sysfs write
<code>net_get_rx_timestamp(msg)</code>	clock_gettime	software time
<code>SO_TIMESTAMPING cmsg</code>		
<code>rt_thread_setup()</code>	no-op	SCHED_FIFO + CPU_SET(3) + mlockall

Everything else — PI math, packet parsing, state machine, virtual clock — is identical on both platforms.

18.4 What Unit Tests Cover (WSL2, no hardware needed)

Test file	Covers
<code>test_protocol.c</code>	serialize↔deserialize round-trip, CRC32, endian byte order, magic/version, padding zeroed
<code>test_clock.c</code>	Q32.32 multiply, <code>read_global_time()</code> formula, monotonicity
<code>test_sync.c</code>	offset/delay from T1–T4 fixtures, min-delay filter accept/reject, PI step vs slew decision, integrator clamp
<code>test_election.c</code>	state transitions, holdover timeout, lower-ID demotion trigger

18.5 What RPi Tests Cover

- SCHED_FIFO priority 85 on Core 3: verified via `/proc/<pid>/stat`
 - SO_TIMESTAMPING kernel timestamps present in cmsg
 - GPIO 18 pulse toggling at ~1 Hz (self-test via GPIO input read loop)
 - GPIO 23 transitions HIGH on startup → LOW after convergence (single-node: LEADER, offset ≈ 0)
 - Telemetry UDP packets arriving on Windows listener
 - Memory locked: `VmLck > 0` in `/proc/<pid>/status`
-

18.6 Telemetry Design

On RPi (non-RT drain thread):

- RT sync thread writes fixed-size records to a lock-free ring buffer (power-of-2, atomic head/tail)
- Separate low-priority drain thread sends records via UDP to the Windows machine on port 4242
- Ring buffer write is a single compare-and-exchange — no blocking, no allocation

On Windows (scripts/telemetry_listen.py):

- `socket.recvfrom()` loop prints CSV: `timestamp_ns, state, offset_ns, rtt_ns, rate_q32`
 - Run in WSL2 or native Python; output piped to a file for analysis
-

18.7 Build Commands (run from Windows via `wsl ...`)

18.7.1 Inner loop

```
wsl cmake -S . -B build-host -DPLATFORM=host
wsl cmake --build build-host
wsl ./build-host/tests/drs_tests
```

18.7.2 Outer loop

```
# Cross-compile
wsl cmake -S . -B build-arm -DCMAKE_TOOLCHAIN_FILE=toolchain-aarch64.cmake
wsl cmake --build build-arm

# Deploy
wsl bash scripts/deploy.sh <rpi-ip>

# Capture telemetry for 30 s
wsl python3 scripts/telemetry_listen.py --duration 30
```

18.8 Implementation Order & Commit Strategy

Each step below ends with a commit **before moving to the next step**. Commits are made as soon as the step's tests pass — not batched at the end. One logical unit per commit (one module, or one tightly related pair of files). Planning documents and workflow files never enter git history.

1. `CMakeLists.txt` + `toolchain-aarch64.cmake` + minimal tests/ scaffold → confirm WSL2 builds → **commit**
2. `.github/workflows/ci.yml` — GitHub Actions: build host target, run tests, cross-compile ARM → CI green on GitHub → **commit**
3. `protocol.h/c` + `test_protocol.c` → all protocol tests pass → **commit**
4. `clock.h/c` + `test_clock.c` → all clock tests pass → **commit**
5. `sync.h/c` + `test_sync.c` → all sync tests pass → **commit**
6. `election.h/c` + `test_election.c` → all election tests pass → **commit**
7. `net.h/c` (with host stub) → compiles on host → **commit**
8. `gpio.h/c` (with host stub) → compiles on host → **commit**

9. `telemetry.h/c` → compiles on host → **commit**
 10. `main.c` → full binary links and runs on host → **commit**
 11. `scripts/deploy.sh` + `scripts/telemetry_listen.py` → **commit**
 12. RPi hardware validation (no new code commits unless fixes needed)
-

18.9 Verification Checklist

- [] `drs_tests` reports 0 failures across all 4 suites
- [] `ssh pi "cat /proc/$(pgrep drs_sync)/status | grep VmLck"` → nonzero
- [] `ssh pi "cat /proc/$(pgrep drs_sync)/task/*/stat"` → CPU column shows 3
- [] Telemetry stream shows `state=LEADER`, `offset_ns` near 0
- [] GPIO 23 transitions LOW within 10 s of startup